# tmux + VSCode Terminal Setup

## Per-Agent Session Isolation

**Topic:** `tmux-vscode-setup` **Status:** Live — in use across IDEA agent workspaces **Date:** 2026-03-20

---

## The Problem

Each IDEA agent runs in its own workspace folder and has its own Claude Code session. Without session isolation, opening a VSCode terminal drops you into a plain shell — no Claude, no preserved history, and no way to distinguish which agent you are working with.

The goal: opening the integrated terminal in any agent workspace automatically connects to that agent's named tmux session, with Claude already running (or resuming) inside it.

---

## How It Works — Overview

```
VSCode opens workspace at:  /home/pi/idea/agents/agent-<role>/
        │
        ▼
.vscode/settings.json defines a custom terminal profile named "tmux"
        │
        ▼
On terminal open: bash -c "tmux attach -t <session> 2>/dev/null || tmux new-
session -s <session> 'claude; exec bash -l'"
        │
        ├─ Session exists  →  attaches (Claude resumes in existing session)
        └─ No session yet  →  creates new session and starts Claude
```

The key insight is that VSCode's `terminal.integrated.profiles.linux` accepts a `path` + `args` that can run any shell command — so we make it run a tmux attach/create idiom instead of a plain shell.

---

## Component 1 — Global tmux Configuration

File: `/home/pi/.tmux.conf`

```
set mouse on

# Resize each window to the largest attached client
# (prevents locking to the 24×80 default when a new client attaches)
setw -g aggressive-resize on
```

```
# Force resize to current client dimensions on every attach
# (makes the session scale correctly to the VSCode window size)
set-hook -g client-attached 'resize-window -A'

# Generous scrollback buffer
set -g history-limit 50000

# Allow Shift+drag to bypass tmux mouse capture for normal terminal copy
set -g terminal-overrides 'xterm*:smcup@:rmcup@'
```

### Why each setting matters

| Setting | Why |
|---|---|
| `mouse on` | Click to select pane, scroll with the wheel |
| `aggressive-resize` | Without this, the session stays locked to the smallest attached client's dimensions — opening a second terminal shrinks everything |
| `client-attached resize-window -A` | Forces the window to match the newly attached VSCode terminal's actual size |
| `history-limit 50000` | Long Claude sessions produce a lot of output; the default 2000 lines loses context quickly |
| `terminal-overrides xterm*:smcup@:rmcup@` | Strips alternate-screen mode so that Shift+highlight works for native copy without entering tmux copy-mode |

## Component 2 — Per-Workspace VSCode Settings

Each agent workspace has its own `.vscode/settings.json`. The only tmux-relevant keys are the terminal profile definition.

### Template

```
{
    "terminal.integrated.defaultProfile.linux": "tmux",
    "terminal.integrated.profiles.linux": {
        "tmux": {
            "path": "bash",
            "args": ["-c", "tmux attach -t <SESSION_NAME> 2>/dev/null || tmux
new-session -s <SESSION_NAME> 'claude; exec bash -l'"]
        }
    },
    "terminal.integrated.scrollback": 10000,
```

```
    "terminal.integrated.inheritEnv": true
}
```

Replace `<SESSION_NAME>` with the agent's designated session name (see table below).

### How the shell command works

```
tmux attach -t <SESSION_NAME> 2>/dev/null || tmux new-session -s
<SESSION_NAME> 'claude; exec bash -l'
```

| Part | Meaning |
|---|---|
| `tmux attach -t <SESSION_NAME>` | Try to attach to an existing named session |
| `2>/dev/null` | Suppress the "no sessions" error if it doesn't exist yet |
| `\|\|` | If attach fails (exit code ≠ 0), run the right side |
| `tmux new-session -s <SESSION_NAME> 'claude; exec bash -l'` | Create a new session and immediately start `claude`; `exec bash -l` falls back to a login shell if Claude exits |

### Extra keys (optional but recommended)

| Key | Value | Why |
|---|---|---|
| `terminal.integrated.scrollback` | `10000` | VSCode's own scrollback buffer (in addition to tmux's) |
| `terminal.integrated.gpuAcceleration` | `"off"` | Prevents rendering artefacts in some Raspberry Pi GPU driver combinations |
| `terminal.integrated.inheritEnv` | `true` | Ensures PATH and other env vars set by your shell profile are available inside the session |

---

## Component 3 — Session Naming Convention

Session names follow the pattern: **`claude-<agent-slug>`**

| Agent | Role | Session name |
|---|---|---|
| Compass | researcher | `claude-agent-researcher` |
| Axle | engine-dev | `claude-engine` |
| Pixel | console-dev | `claude-console` *(not yet configured)* |
| Beacon | site-dev | `claude-site` *(not yet configured)* |

| Agent | Role | Session name |
|---|---|---|
| Veri | quality-manager | `claude-quality` *(not yet configured)* |
| Marco | programme-manager | `claude-programme` *(not yet configured)* |

> **Note:** `agent-engine-dev` currently uses `claude-engine` (shorter form). Prefer the shorter `claude-<role>` form for new agents; it is easier to type. The researcher uses the longer `claude-agent-researcher` form for historical reasons.

---

## Current State

| Workspace | `.vscode/settings.json` exists | tmux profile configured |
|---|---|---|
| `agent-researcher` | Yes | Yes — session `claude-agent-researcher` |
| `agent-engine-dev` | Yes | Yes — session `claude-engine` |
| `agent-console-dev` | No | No |
| `agent-site-dev` | No | No |
| `agent-quality-manager` | No | No |
| `agent-programme-manager` | No | No |

---

## Setup Steps for a New Agent Workspace

1. **Create** `.vscode/settings.json` in the agent workspace root.
2. **Paste the template** from Component 2 above, substituting the session name.
3. **Reload the VSCode window** (Cmd/Ctrl+Shift+P → "Developer: Reload Window") or close and reopen the workspace.
4. **Open a new terminal** (Ctrl+`). VSCode will use the `tmux` profile by default, attaching to or creating the named session.
5. **Verify:** `tmux list-sessions` should show the new session name.

### One-liner to create a settings file from the command line

```
SESSION="claude-<role>"
mkdir -p .vscode
```

```
cat > .vscode/settings.json << EOF
{
    "terminal.integrated.defaultProfile.linux": "tmux",
    "terminal.integrated.profiles.linux": {
        "tmux": {
            "path": "bash",
            "args": ["-c", "tmux attach -t $SESSION 2>/dev/null || tmux new-
session -s $SESSION 'claude; exec bash -l'"]
        }
    },
    "terminal.integrated.scrollback": 10000,
    "terminal.integrated.inheritEnv": true
}
EOF
```

---

## Common Issues

### “no current session” error on startup

This appears when the VSCode terminal launches before any tmux server is running (e.g. after a reboot). It is harmless — the `||` branch creates a new session immediately. The error is printed to the terminal momentarily and then disappears as the new session opens.

To eliminate it entirely, suppress stderr: `2>/dev/null` is already present in the command template.

### Session is the wrong size (tiny or huge)

Caused by `aggressive-resize` not being set, or a stale client with different dimensions still attached. Fix: ensure `/home/pi/.tmux.conf` contains both `aggressive-resize on` and the `client-attached` resize hook. Then detach any stale clients: `tmux detach-client -a` (detach all but the current one).

### Claude doesn’t start automatically

Check the `tmux new-session` command in `.vscode/settings.json` — the shell command string must be exactly `'claude; exec bash -l'`. If `claude` is not on PATH inside tmux, add a full path or source your profile first:

```
tmux new-session -s <SESSION> 'bash -l -c "claude; exec bash -l"'
```

### GPUAcceleration rendering glitches

Add `"terminal.integrated.gpuAcceleration": "off"` to `.vscode/settings.json`.

---

## Viewing Session History and Memory

## Are there viewers?

There is no built-in UI for browsing Claude Code session history. The files are readable but raw:

- **Memory files** — plain `.md` files; open directly in any editor or markdown viewer
- **Session JSONL** — one JSON object per line; no native viewer, but easily converted

## Where the files live

| What | Path |
|---|---|
| Memory files | `~/.claude/projects/<path-hash>/memory/*.md` |
| Session conversation logs | `~/.claude/projects/<path-hash>/<session-uuid>.jsonl` |
| Global input history | `~/.claude/history.jsonl` |

The `<path-hash>` is the workspace path with `/` replaced by `-` (e.g. `/home/pi/idea/agents/agent-researcher` → `-home-pi-idea-agents-agent-researcher`).

## Converting a session to readable markdown

The JSONL format contains records of type `user`, `assistant`, `progress`, `system`, and `file-history-snapshot`. Only `user` and `assistant` records carry conversation content. Assistant content is a list of typed blocks: `thinking`, `text`, and `tool_use`.

A minimal Python converter:

```
import json
from datetime import datetime

SESSION_FILE = '/home/pi/.claude/projects/-home-pi-idea-agents-agent-
researcher/<uuid>.jsonl'

def fmt_ts(ts):
    try:
        return
datetime.fromisoformat(ts.replace('Z','+00:00')).strftime('%H:%M:%S')
    except:
        return ts or ''

def extract_text(content):
    if isinstance(content, str):
        return content.strip()
    parts = []
    for c in content:
        if c.get('type') == 'text' and c.get('text','').strip():
            parts.append(c['text'].strip())
        elif c.get('type') == 'tool_use':
```

```
            parts.append(f"`[tool: {c['name']}]`")
    return '\n\n'.join(parts)

with open(SESSION_FILE) as f:
    for line in f:
        obj = json.loads(line.strip())
        if obj.get('type') not in ('user', 'assistant'):
            continue
        role = obj['type']
        ts = fmt_ts(obj.get('timestamp',''))
        content = obj.get('message', {}).get('content', '')
        text = extract_text(content) if isinstance(content, list) else
content.strip()
        if text:
            prefix = '**You**' if role == 'user' else '**Compass**'
            print(f'### {prefix} `[{ts}]`\n\n{text}\n')
```

To find the right `<uuid>`: list `~/.claude/projects/<path-hash>/` sorted by modification time — the most recently changed `.jsonl` is the current session.

### Example output

See `outputs/2026-03-20-1300-session-and-memory-export.md` for an export of today's session (2026-03-20) including both memory files and the full conversation, produced using the converter above.

---

## Files Reference

| File | Purpose |
|---|---|
| `/home/pi/.tmux.conf` | Global tmux options (mouse, resize, scrollback, copy) |
| `<workspace>/.vscode/settings.json` | Per-workspace VSCode terminal profile (tmux attach/create command) |

---

## Claude Code Session Context

### What is a "session"?

A Claude Code session is an **in-memory conversation**. It is the running `claude` process. As long as that process is alive, Claude holds the full conversation context in memory — every message you sent, every response it gave, every tool call and its output.

When `claude` exits, that context is gone. Nothing is automatically reloaded when you start `claude` again.

### What tmux actually provides

tmux does not give Claude any special context. What it does is **keep the process alive**.

Without tmux: closing the VSCode terminal window kills the shell, which kills `claude`. Context lost.

With tmux: the terminal window is just a view into the tmux session. Closing VSCode detaches from the session but the `claude` process keeps running inside it. Re-opening the terminal re-attaches to the same running process with its full in-memory context intact.

This is process persistence, not session resumption.

## Where things are stored on disk

Claude Code writes several artefacts to `~/.claude/` during a session. These survive process exit but are **not reloaded** into a new `claude` process automatically.

| What | Location | Persists after exit? | Reloaded on restart? |
|---|---|---|---|
| User inputs (your messages) | `~/.claude/history.jsonl` | Yes — global append-only log | No |
| Full conversation (messages + Claude responses) | `~/.claude/projects/<path-hash>/<session-uuid>/subagents/*.jsonl` | Yes | No |
| Tool call outputs (large results) | `~/.claude/projects/<path-hash>/<session-uuid>/tool-results/<tool-id>.txt` | Yes | No |
| Project memory | `~/.claude/projects/<path-hash>/memory/MEMORY.md` | Yes | **Yes** — loaded at startup |
| File edit history | `~/.claude/file-history/<session-uuid>/` | Yes | No |
| Plans | `~/.claude/plans/*.md` | Yes | No |
| Session metadata | `~/.claude/usage-data/session-meta/<session-uuid>.json` | Yes | No |

The one thing that **is** reloaded on every new `claude` process: **project memory** (`MEMORY.md` and any files listed in it) and **`CLAUDE.md`** from the workspace. Everything else is written as an audit trail but not fed back to Claude.

## What “context” means in practice

When you open a terminal in an agent workspace and `claude` is already running in the tmux session, the context Claude has is:

- Every message and response from the current process start (in memory)
- The contents of `CLAUDE.md` (loaded at startup)

- The contents of any memory files (loaded at startup)
- Any files it has read during the current conversation

When you open a terminal and `claude` is **not** running (first start, or after a reboot):

- A fresh process starts with no conversation history
- `CLAUDE.md` is loaded
- Memory files are loaded
- Nothing else — a clean slate within the project's established identity

### Implications for the IDEA agent setup

Each named tmux session (`claude-agent-researcher`, `claude-engine`, etc.) maps to one running `claude` process in one workspace. As long as the session stays alive:

- The agent accumulates conversational context across multiple interactions in the same process
- No context is lost between one VSCode terminal close and the next open

When the session dies (reboot, crash, manual kill):

- The agent starts fresh from CLAUDE.md and memory files only
- Conversational history from before the restart is not recoverable by Claude (though the JSONL files remain on disk for human review)
- This is why important decisions and facts belong in **memory files**, not just in conversational history

The per-agent memory system (writing to `memory/MEMORY.md` and linked files) is specifically designed to bridge this gap: it captures what matters from conversation into a form that survives process restarts.

---